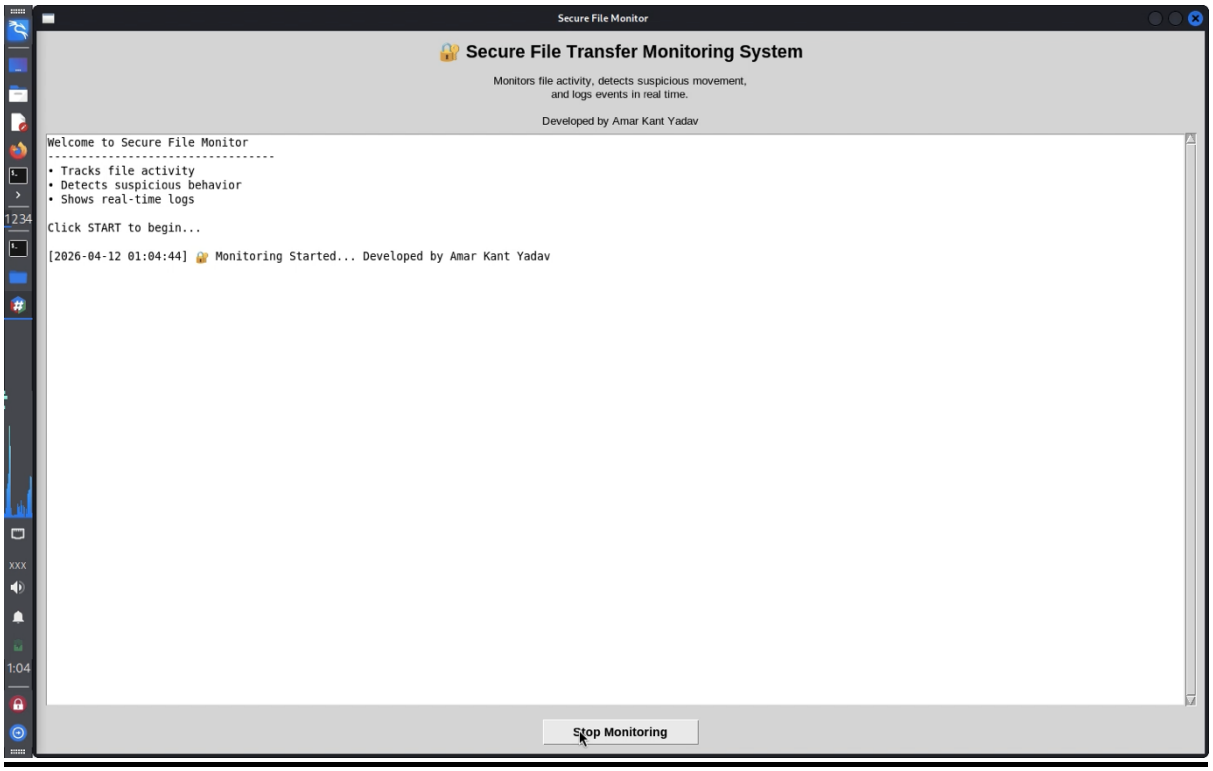
Secure File Transfer Monitoring System

Detailed Security Analysis, Architecture & Implementation Report

This project focuses on developing a Secure File Transfer Monitoring System designed to ensure data confidentiality and track file movement across a system or network. File transfers—both internal and external—pose significant risks including data leakage, unauthorized access, malware distribution, and insider misuse.



Abstract

File transfer operations are a critical security concern in modern computing environments. Unauthorized transfers, data exfiltration, and file tampering can lead to severe data breaches. This project presents a Secure File Transfer Monitoring System that continuously monitors file system activity, detects suspicious behavior, verifies file integrity using cryptographic hashing, and generates audit-ready reports. The system follows a defensive security model inspired by real-world endpoint monitoring and data loss prevention (DLP) systems.

1. Introduction

1.1 Background

File systems are frequently targeted by attackers and insiders to move, modify, or exfiltrate sensitive data. Common threats include unauthorized file transfers, data leakage, and malware manipulation of files.

1.2 Project Overview

The Secure File Transfer Monitoring System is a real-time monitoring tool designed to observe file activities such as creation, modification, deletion, and movement. It uses intelligent detection techniques to identify suspicious activities and generate detailed logs.

1.3 Practical Motivation

Manual monitoring of file systems is inefficient and prone to human error. This project automates monitoring and provides consistent detection, enabling better defensive strategies.

2. Project Objectives

- Monitor file system activities in real time
- Detect unauthorized file movement
- Perform integrity verification using SHA256 hashing
- Generate alerts for suspicious behaviour
- Produce structured audit reports

Simulate real-world DLP and EDR functionality

3. Tools, Technologies & Environment

3.1 Programming Language

Python 3 is used for its simplicity, cross-platform compatibility, and strong library support.

3.2 Libraries Used

- watchdog – monitors file system events
- hashlib – performs cryptographic hashing
- tkinter – provides GUI interface
- reportlab – generates PDF reports

3.3 Operating Environment

The system is designed to run on Windows, Linux, and macOS without requiring administrative privileges.

4. System Architecture

The system follows a layered architecture consisting of multiple components working together:

- GUI Layer: Provides user interaction and real-time logs

```
    # ================ GUI ================

    class App:
        def __init__(self, root):
            self.root = root
            self.root.title("Secure File Monitor")
            self.root.geometry("800x550")

            root.rowconfigure(1, weight=1)
            root.columnconfigure(0, weight=1)

            # Title
```

```
            top = tk.Frame(root)
            top.grid(row=0, column=0, sticky="ew")

            tk.Label(top,
                     text="🔐 Secure File Transfer Monitoring System",
                     font=("Arial", 16, "bold")).pack(pady=10)

            tk.Label(top,
                     text="Monitors file activity, detects suspicious
movement,\nand logs events in real time.\n\nDeveloped by Amar Kant Yadav",
                     font=("Arial", 10)).pack()

            # Log box
            self.log_box = scrolledtext.ScrolledText(root)
            self.log_box.grid(row=1, column=0, sticky="nsew", padx=10, pady=5)

            self.log_box.insert(tk.END,
                "Welcome to Secure File Monitor\n"
                "-----------------------------------\n"
                "• Tracks file activity\n"
                "• Detects suspicious behavior\n"
                "• Shows real-time logs\n\n"
                "Click START to begin...\n\n"
            )

            # Button
            bottom = tk.Frame(root)
            bottom.grid(row=2, column=0, sticky="ew", pady=10)

            self.button = tk.Button(bottom,
                                    text="Start Monitoring",
                                    bg="green",
                                    fg="white",
                                    width=20,
                                    font=("Arial", 11, "bold"),
                                    command=self.toggle)

            self.button.pack()

        def toggle(self):
            global running

            if not running:
                self.button.config(text="Stop Monitoring", bg="red")
                threading.Thread(target=start_monitor, daemon=True).start()
            else:
                self.button.config(text="Start Monitoring", bg="green")
                stop_monitor()
```

- Monitoring Engine: Captures file system events

```python
# ================ MONITOR ================

def start_monitor():
    global observer, running

    if running:
        return

    running = True
    observer = Observer()

    paths = [
        os.path.expanduser("~/Desktop"),
        os.path.expanduser("~/Documents"),
        os.path.expanduser("~/Downloads")
    ]

    for p in paths:
        if os.path.exists(p):
            observer.schedule(Handler(), p, recursive=True)

    observer.start()
    log_event("🔐 Monitoring Started... Developed by Amar Kant Yadav")

def stop_monitor():
    global observer, running

    if observer:
        observer.stop()
        observer.join()

    running = False
    log_event("⛔ Monitoring Stopped")

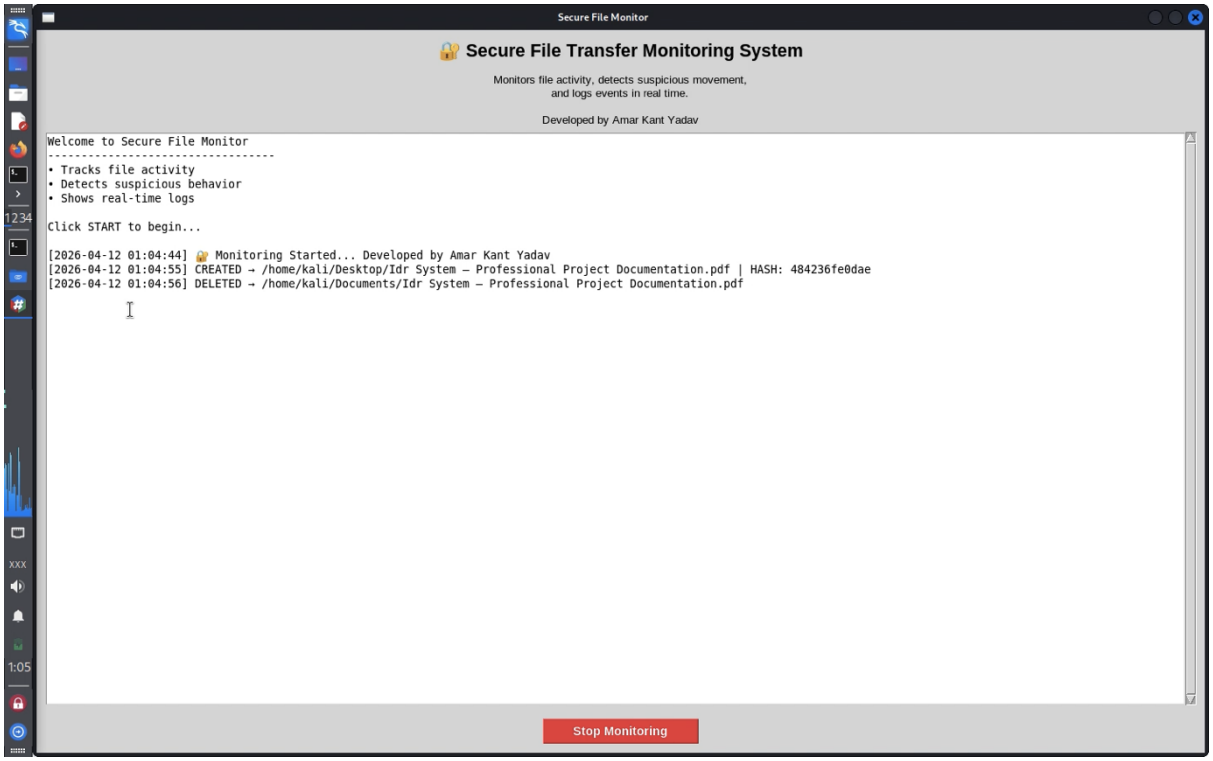
    generate_report()
```

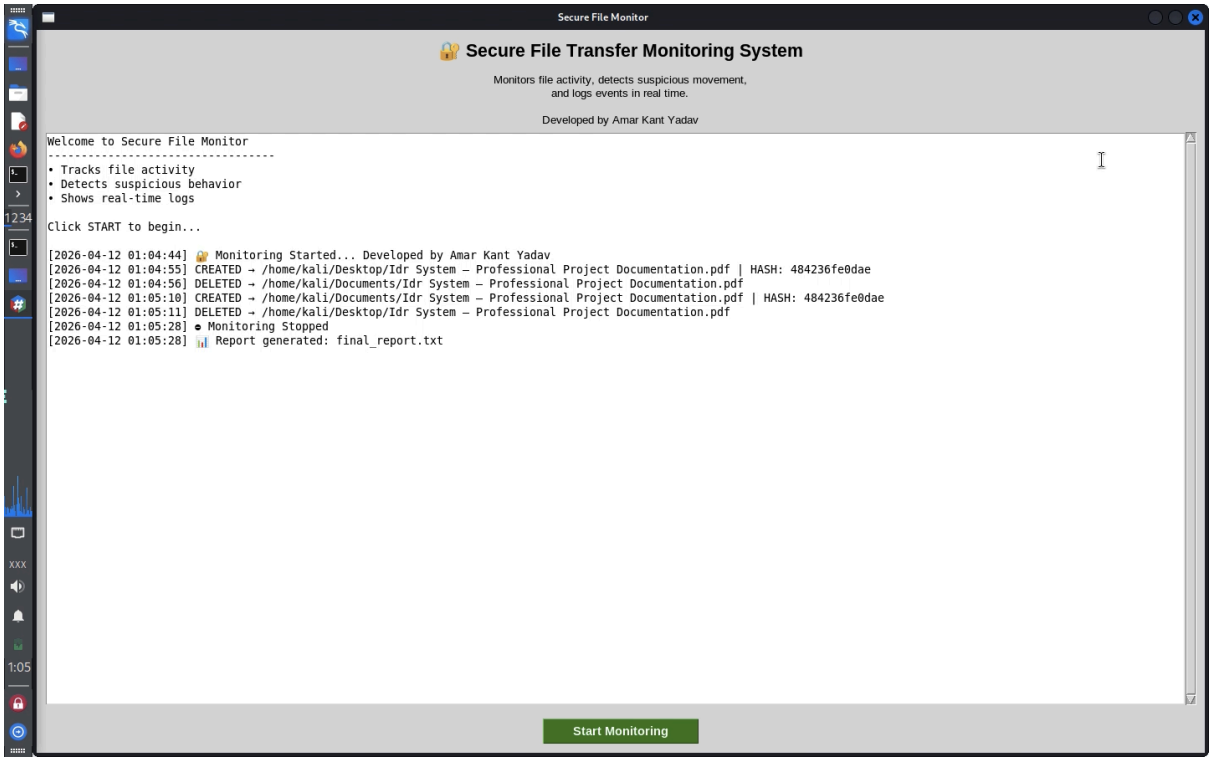
- Detection Engine: Analyzes events and detects anomalies
- Integrity Module: Verifies file integrity using hashing

Logging & Reporting Module: Stores logs and generates reports

5. Workflow & Execution Flow

START → Initialize GUI → Start Monitoring → Capture File Events → Analyze Events → Perform Hashing → Detect Suspicious Activity → Log Events → Generate Report → END





6. Detailed Module Implementation

6.1 File Monitoring Module

Tracks file system events such as creation, deletion, modification, and movement using watchdog observers.

6.2 Integrity Verification Module

Uses SHA256 hashing to ensure that files are not tampered with during transfers.

6.3 Detection Engine

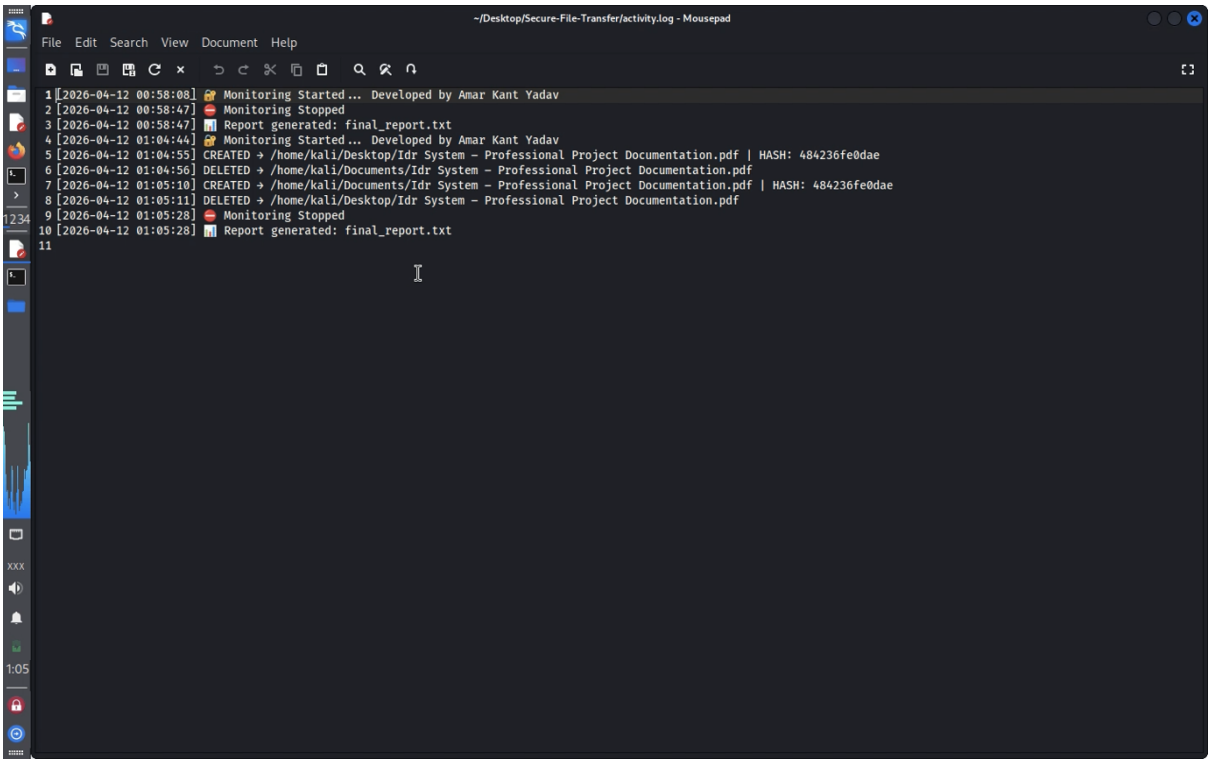
Analyzes file events to detect suspicious activities such as unauthorized movement or bulk transfers.

6.4 Smart Move Detection

Implements time-based correlation to reconstruct file movement events from create and delete operations.

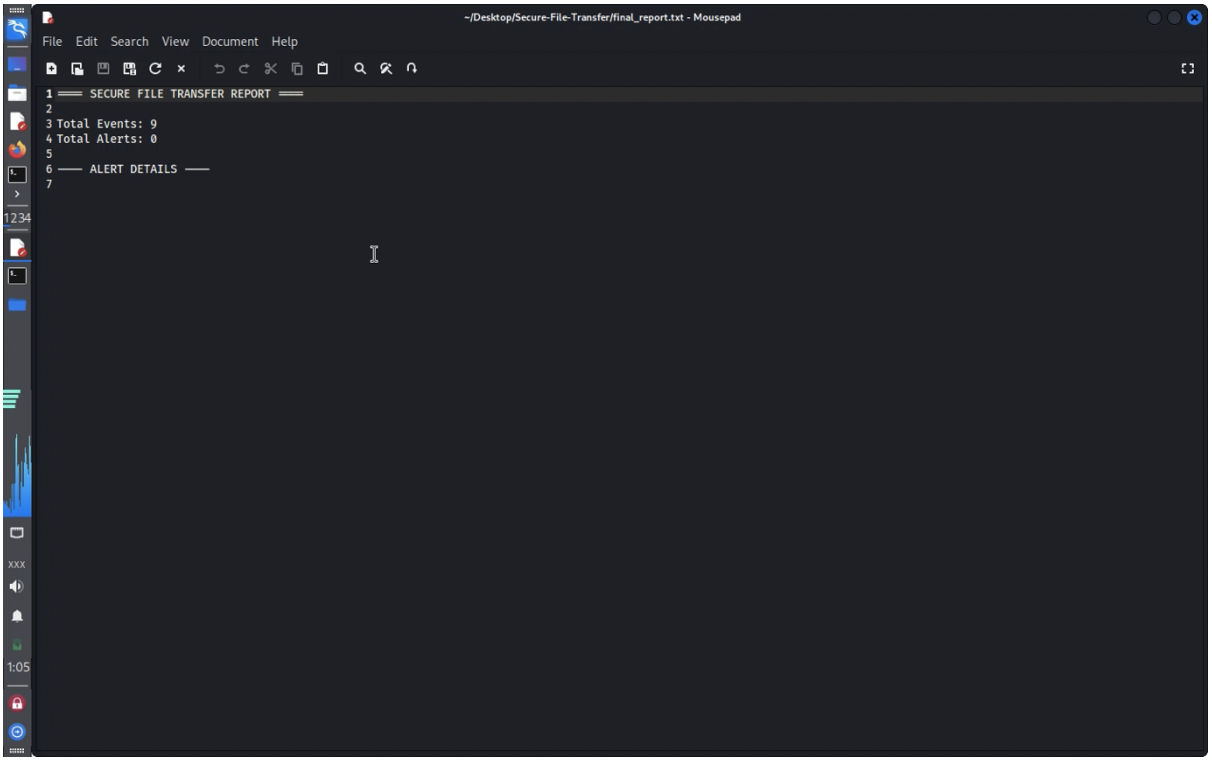
6.5 Logging Module

Logs all file activities in real time and stores them in a persistent log file.



6.6 Reporting Module

Generates structured reports summarizing file activities and detected alerts.



7. Results & Observations

The system successfully monitored file operations and detected movements accurately. Smart detection logic reduced false positives and improved accuracy.

8. Learning Outcomes

- Understanding of file system monitoring
- Knowledge of data loss prevention systems
- Experience with hashing and integrity verification
- Real-time logging techniques
- Security-focused software design

9. Conclusion

The Secure File Transfer Monitoring System provides a practical and effective approach to monitoring file activities. It

enhances security visibility and supports defensive cybersecurity practices.

10. Disclaimer

This project is intended for educational and defensive purposes only. No exploitation is performed.

11. References

- Python Documentation
- Watchdog Library
- MITRE ATT&CK; Framework
- Cybersecurity Best Practices